Übung 4 Musterlösung

JavaScript (JS)

Einführung in Webengineering · WS 2026/27

Die Musterlösung erläutert alle acht Aufgaben mit Ausgaben, Begründungen und Beispielcode. Andere Implementierungen sind möglich, wenn sie die jeweiligen Anforderungen erfüllen.

01 JavaScript ausführen und Typen untersuchen

a) HTML und JavaScript verbinden

Die Datei `kapitel04/index.html` bindet die JavaScript-Datei am Ende von `body` ein:

```
<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <title>JavaScript üben</title>
</head>
<body>
  <h1>JavaScript üben</h1>
  <script src="aufgabe01.js"></script>
</body>
</html>
```

In `kapitel04/aufgabe01.js` steht `console.log("Hallo JavaScript!");`. Nach dem Öffnen der HTML-Datei erscheint die Begrüßung in der Browserkonsole.

b) Typen und dynamische Typisierung

Die vier Aufrufe geben der Reihe nach folgende Werte aus:

```
"number"    "string"    "boolean"
"undefined" "object"
"object"    "object"
"function"  true
```

`typeof` liefert einen String. Bei `null` ist `"object"` eine historische Besonderheit; `null` ist ein primitiver Wert. Arrays sind Objekte und lassen sich mit `Array.isArray()` erkennen. Funktionen sind aufrufbare Objekte, für die `typeof` den Wert `"function"` liefert.

```
var wert = 7;      console.log(typeof wert); // "number"
wert = "sieben";    console.log(typeof wert); // "string"
wert = [7];         console.log(typeof wert); // "object"
```

Die Variable ist nicht auf einen Datentyp festgelegt. Bei jeder Zuweisung kann sie einen Wert eines anderen Typs erhalten.

c) Besonderheiten von Number

```
console.log(0.1 + 0.2 === 0.3); // false
console.log(1 / 0);             // Infinity
console.log(Number("Web"));     // NaN
console.log(typeof NaN);        // "number"
```

`Number` verwendet binäre Gleitkommazahlen. `0.1` und `0.2` sind darin nicht exakt darstellbar; ihre Summe ergibt `0.30000000000000004`. `Infinity` steht hier für das Ergebnis der Division durch null. Die fehlgeschlagene Zahlumwandlung ergibt `NaN`; beide Sonderwerte gehören zum Typ `number`.

02 Typumwandlungen und Vergleiche verstehen

a) Ergebnisse und Umwandlungen

```
console.log("5" + 2);               // "52"   (string)
console.log("5" - 2);               // 3      (number)
console.log(Number("5") + 2);       // 7      (number)
console.log(0 == "0");              // true   (boolean)
console.log(0 === "0");             // false  (boolean)
console.log(null == undefined);     // true   (boolean)
console.log(null === undefined);    // false  (boolean)
```

Beim ersten Ausdruck verkettet `+` Strings; die Zahl `2` wird dafür implizit zu `"2"`. Bei der Subtraktion wird `"5"` implizit zur Zahl `5`. `Number("5")` führt diese Umwandlung ausdrücklich aus.

`==` gleicht beim Vergleich von `0` mit `"0"` die Typen an. Für `null == undefined` gilt eine besondere Gleichheitsregel. `===` führt keine Typumwandlung durch; unterschiedliche Typen ergeben in diesen Beispielen `false`.

b) Truthy und falsy

`Boolean(0)` und `Boolean("")` ergeben `false`. `Boolean("0")`, `Boolean(" ")`, `Boolean([])` und `Boolean({})` ergeben `true`. Ein leerer String enthält kein Zeichen; ein Leerzeichen ist bereits ein Zeichen. Auch leere Arrays und Objekte sind truthy.

`===` vermeidet überraschende Gleichheit durch implizite Typumwandlungen. Wenn eine Umwandlung beabsichtigt ist, kann sie vor dem Vergleich ausdrücklich erfolgen.

03 Parameter und Funktionen als Werte verwenden

a) Fehlende Angabe und Rabatt von 0 Prozent unterscheiden

Die ursprüngliche Funktion liefert `90`, `90` und `80`. `||` wählt den rechten Operanden, wenn der linke falsy ist. Daher werden sowohl `undefined` als auch die gültige Angabe `0` durch `10` ersetzt. Die Korrektur prüft nur auf `undefined`:

```
function preis(betrag, rabatt) {
  if (rabatt === undefined) {
    rabatt = 10;
  }
  return betrag * (1 - rabatt / 100);
}
console.log(preis(100));      // 90
console.log(preis(100, 0));   // 100
console.log(preis(100, 20)); // 80
```

b) Eine Funktion zweimal anwenden

```
function zweimal(f, x) {
  return f(f(x));
}
function plusFuenf(x) { return x + 5; }
function quadrat(x) { return x * x; }
console.log(zweimal(plusFuenf, 10)); // 20
console.log(zweimal(quadrat, 3));    // 81
```

Die Zwischenschritte sind `10 → 15 → 20` und `3 → 9 → 81`. Der Parameter `f` erhält einen Funktionswert, den `zweimal` selbst aufrufen kann. Ein bereits berechnetes Zahlenergebnis wäre nicht aufrufbar.

04 Scope und Hoisting nachvollziehen und Fehler finden

a) Lokale und äußere Variable

Die Ausgaben lauten in dieser Reihenfolge:

```
undefined
"innen"
"außen"
```

Die Deklaration `var wert` in `pruefen()` gilt im gesamten Funktionskörper und verdeckt dort die äußere Variable. Beim Eintritt in die Funktion ist die lokale Variable bereits vorhanden und zunächst `undefined`. Die Zuweisung von `"innen"` geschieht erst an ihrer Stelle im Programmablauf.

Deshalb liest die erste Ausgabe den noch nicht zugewiesenen lokalen Wert. Die zweite liest `"innen"`. Außerhalb der Funktion bleibt die äußere Variable unverändert bei `"außen"`. Hoisting beschreibt dieses Verhalten; der Quelltext wird dabei nicht tatsächlich umsortiert.

Zur Kontrolle setzen Sie im Quelltextbereich der Entwicklerwerkzeuge einen Haltepunkt auf die erste Zeile `console.log(wert)` innerhalb von `pruefen()`. Nach dem Neuladen hält der Debugger vor dieser Ausgabe an. Im lokalen Gültigkeitsbereich steht `wert` auf `undefined`. Nach Ausführung der Zuweisung steht dort `"innen"`.

b) Funktionsdeklaration und Funktionsausdruck

Versuch 1 gibt `"Start"` aus. Die Funktionsdeklaration stellt die Funktion bereits bereit, bevor die erste Anweisung ausgeführt wird:

```
start(); // "Start"
function start() { console.log("Start"); }
```

Versuch 2 führt zu einem `TypeError`: Die mit `var` deklarierte Variable `start` existiert bereits, hat vor der Zuweisung aber den Wert `undefined`. Dieser Wert ist nicht aufrufbar. Der Fehler beendet die Ausführung des Codeblocks, bevor die Funktionszuweisung erreicht wird.

Beim Funktionsausdruck muss der Aufruf nach der Zuweisung erfolgen:

```
var start = function () { console.log("Start"); };
start(); // "Start"
```

Beide Versuche müssen getrennt ausgeführt werden, damit eine vorhandene Funktion namens `start` das Ergebnis nicht beeinflusst.

c) Zeilenumbruch direkt nach return

Die ursprüngliche Funktion liefert `undefined`. Der Zeilenumbruch unmittelbar nach `return` bewirkt hier die automatische Semikolon-Einfügung: Die Anweisung wirkt wie `return;` und beendet die Funktion ohne Rückgabewert. Die öffnende Klammer muss auf derselben Zeile wie `return` stehen:

```
function daten() {
  return { kurs: "JS" };
}
console.log(daten()); // { kurs: "JS" }
```

05 Objekte teilen und von JSON unterscheiden

a) Gemeinsames Objekt und neue Parameterzuweisung

Der Aufruf `console.log(kurs.raum, kopie === kurs)` gibt `"B202"` und `true` aus. `var kopie = kurs` erzeugt kein neues Objekt. Beide Variablen verweisen auf dasselbe Objekt.

Beim Funktionsaufruf erhält `obj` eine Kopie dieses Referenzwerts. Daher verändert `obj.raum = "B202"` das gemeinsam verwendete Objekt; die Änderung ist anschließend auch über `kurs` und `kopie` sichtbar.

Die Anweisung `obj = { titel: "CSS", raum: "C303" }` bindet nur den lokalen Parameter an ein neues Objekt. Sie ändert weder `kurs` noch `kopie`. Auch bei Objekten werden Argumentwerte übergeben; der übergebene Wert ist hier eine Referenz.

b) Array und Methode ergänzen

Ein eigenständig ausführbares Beispiel mit dem Objektzustand nach Teil a):

```
var kurs = { titel: "JavaScript", raum: "B202" };
kurs.termine = ["Montag", "Mittwoch"];
kurs.beschreibung = function () {
  return this.titel + " in Raum " + this.raum;
};
console.log(kurs.beschreibung()); // "JavaScript in Raum B202"
kurs.raum = "D404";
console.log(kurs.beschreibung()); // "JavaScript in Raum D404"
```

Die Methode liest die Eigenschaften bei jedem Aufruf erneut. Dadurch erscheint nach der Raumänderung der aktuelle Wert. Beim Aufruf `kurs.beschreibung()` verweist `this` auf `kurs`.

c) Die Daten als JSON

Nach der weiteren Raumänderung in Teil b) sind die Daten beispielsweise so darstellbar:

```
{
  "titel": "JavaScript",
  "raum": "D404",
  "termine": ["Montag", "Mittwoch"]
}
```

In JSON stehen Eigenschaftsnamen und Stringwerte in doppelten Anführungszeichen. Zwischen Einträgen stehen Kommas; nach dem letzten Eintrag folgt kein zusätzliches Komma. Die eckigen Klammern kennzeichnen das Array.

Ein JavaScript-Objekt kann Funktionen als Eigenschaftswerte besitzen. JSON beschreibt Daten und kennt keinen Funktionswert. Die Methode `beschreibung` wird deshalb hier nicht übernommen. Auch Variablendeklarationen und Kommentare gehören nicht zum JSON-Text.

06 Objekte mit Fabrikfunktionen kapseln

a) Fabrikfunktion

```
function erzeugeKurs(titel) {
  return {
    beschreibung: function () {
      return "Kurs: " + titel;
    }
  };
}
```

Jeder Aufruf erzeugt ein neues Objekt. Die Methode liest den Parameter `titel` des jeweiligen Fabrikaufrufs. Das zurückgegebene Objekt besitzt zunächst nur die Methode `beschreibung`.

b) Ausgaben und Kapselung

```
var js = erzeugeKurs("JavaScript");
var css = erzeugeKurs("CSS");
console.log(js.beschreibung());  // "Kurs: JavaScript"
console.log(css.beschreibung()); // "Kurs: CSS"
console.log(js.titel);           // undefined
js.titel = "HTML";
console.log(js.beschreibung());  // "Kurs: JavaScript"
```

Die Methode behält Zugriff auf den Gültigkeitsbereich, in dem sie erzeugt wurde. Diese Verbindung heißt Closure. Jeder Fabrikaufruf hat einen eigenen Parameter `titel`. Die neue Eigenschaft `js.titel` ist davon getrennt und ändert dessen Wert nicht.

07 Dynamische Bindung und Klickereignisse verstehen

a) Der Aufruf bestimmt den Wert von this

Die Ausgabe lautet `"Anna"` und `"Ben"`. Bei `anna.vorstellen()` verweist `this` auf `anna`, bei `ben.vorstellen()` auf `ben`. Beide Objekte verwenden dieselbe Funktion. Bei diesen gewöhnlichen Methodenaufrufen bestimmt das Objekt vor dem Punkt den Wert von `this`; der Ort der Funktionsdefinition legt ihn nicht dauerhaft fest.

b) Zwei Schaltflächen

```
<!doctype html>
<html lang="de">
<head><meta charset="utf-8"><title>Schaltflächen</title></head>
<body>
  <button type="button" onclick="markieren(this)">Erste</button>
  <button type="button" onclick="markieren(this)">Zweite</button>
  <script>
    function markieren(element) {
      element.style.backgroundColor = "gold";
    }
  </script>
</body>
</html>
```

Im Attribut `onclick` bezeichnet `this` die angeklickte Schaltfläche. `markieren(this)` übergibt diese als Argument. Der Parameter `element` verweist auf genau diese Schaltfläche, deren Hintergrund geändert wird. Die andere bleibt beim jeweiligen Klick unverändert.

08 Eigenschaften entlang der Prototypenkette finden

a) Prototypenkette und Eigenschaftssuche

Die vollständige Kette lautet:

`jsBuch → buch → medium → Object.prototype → null`

`Object.create(buch)` legt `buch` als Prototyp des neuen Objekts fest. `medium` wurde als gewöhnliches Objektliteral erzeugt und hat deshalb `Object.prototype` als Prototyp. Dessen Prototyp ist `null`; dort endet die Suche.

Gesuchte Eigenschaft	Gefunden auf	Wert
`titel`	`jsBuch`	`"JavaScript"`
`beschreibung`	`buch`	Die gemeinsam verwendete Funktion
`art`	`medium`	`"Lernmaterial"`

Mit den drei Objekten aus der Aufgabenstellung ergibt die Prüfung:

```
console.log(Object.getPrototypeOf(jsBuch) === buch);           // true
console.log(Object.getPrototypeOf(buch) === medium);           // true
console.log(Object.getPrototypeOf(medium) === Object.prototype); // true
console.log(Object.getPrototypeOf(Object.prototype) === null); // true
console.log(jsBuch.beschreibung()); // "JavaScript (Lernmaterial)"
```

Die Methode wird auf `buch` gefunden. Beim Aufruf `jsBuch.beschreibung()` verweist `this` trotzdem auf `jsBuch`. `this.titel` wird dort direkt gefunden, `this.art` erst auf `medium`.

b) Zweites Objekt mit derselben Methode

```
var cssBuch = Object.create(buch);
cssBuch.titel = "CSS";
console.log(cssBuch.beschreibung()); // "CSS (Lernmaterial)"
console.log(jsBuch.beschreibung === cssBuch.beschreibung); // true
```

Beide Zugriffe finden dieselbe Funktion auf `buch`; eine zusätzliche Funktionsinstanz pro Buch entfällt. Die Bücher besitzen eigene Titel und teilen sich die Methode auf ihrem Prototyp.

c) Geerbte Eigenschaft verdecken und wieder sichtbar machen

```
jsBuch.art = "E-Book";
console.log(jsBuch.art);  // "E-Book"
console.log(cssBuch.art); // "Lernmaterial"
console.log(medium.art);  // "Lernmaterial"
delete jsBuch.art;
console.log(jsBuch.art);  // "Lernmaterial"
```

Die Zuweisung legt eine eigene Eigenschaft `art` auf `jsBuch` an. Sie wird bei der Suche zuerst gefunden und verdeckt die gleichnamige Eigenschaft auf `medium`. Das Löschen entfernt nur die eigene Eigenschaft. Danach findet die Suche wieder den geerbten Wert `"Lernmaterial"`.